

# tf.RaggedTensor Stay organized with collections Save and categorize content based on your preferences.

[https://www.tensorflow.org/api\\_docs/python/tf/RaggedTensor](https://www.tensorflow.org/api_docs/python/tf/RaggedTensor)

*Represents a ragged tensor.*

## Code Examples

```
print(tf.RaggedTensor.from_row_splits(  
    values=[3, 1, 4, 1, 5, 9, 2, 6],  
    row_splits=[0, 4, 4, 7, 8, 8]))
```

```
print(tf.RaggedTensor.from_row_splits(  

```

```
    values=[3, 1, 4, 1, 5, 9, 2, 6],
```

```
    row_splits=[0, 4, 4, 7, 8, 8]))
```

```
values = [3, 1, 4, 1, 5, 9, 2, 6]
RaggedTensor.from_row_splits(values, row_splits=[0, 4, 4, 7, 8, 8])

RaggedTensor.from_row_lengths(values, row_lengths=[4, 0, 3, 1, 0])

RaggedTensor.from_value_rowids(
    values, value_rowids=[0, 0, 0, 0, 2, 2, 2, 3], nrows=5)

RaggedTensor.from_row_starts(values, row_starts=[0, 4, 4, 7, 8])

RaggedTensor.from_row_limits(values, row_limits=[4, 4, 7, 8, 8])

RaggedTensor.from_uniform_row_length(values, uniform_row_length=2)
```

```
values = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
RaggedTensor.from_row_splits(values, row_splits=[0, 4, 4, 7, 8, 8])
```

## Documentation

---

Represents a ragged tensor.

### View aliases

Compat aliases for migration

See

Migration guide for  
more details.

`tf.compat.v1.RaggedTensor`

### Used in the notebooks

- Ragged tensors
  - Unicode strings

A `RaggedTensor` is a tensor with one or more ragged dimensions, which are dimensions whose slices may have different lengths. For example, the inner (column) dimension of `rt=[[3, 1, 4, 1], [], [5, 9, 2], [6], []]` is ragged, since the column slices (`rt[0, :]`, ..., `rt[4, :]`) have different lengths.

Dimensions whose slices all have the same length are called uniform dimensions. The outermost dimension of a `RaggedTensor` is always uniform, since it consists of a single slice (and so there is no possibility for differing slice lengths).

The total number of dimensions in a `RaggedTensor` is called its rank, and the number of ragged dimensions in a `RaggedTensor` is called its ragged-rank. A `RaggedTensor`'s ragged-rank is fixed at graph creation time: it can't depend on the runtime values of Tensors, and can't vary dynamically for different session runs.

Note that the `__init__` constructor is private. Please use one of the following methods to construct a `RaggedTensor`:

- `tf.RaggedTensor.from_row_lengths`
- `tf.RaggedTensor.from_value_rowids`
- `tf.RaggedTensor.from_row_splits`
- `tf.RaggedTensor.from_row_starts`
- `tf.RaggedTensor.from_row_limits`
- `tf.RaggedTensor.from_nested_row_splits`
- `tf.RaggedTensor.from_nested_row_lengths`
- `tf.RaggedTensor.from_nested_value_rowids`

## Potentially Ragged Tensors

Many ops support both Tensors and RaggedTensors

(see `tf.ragged` for a full listing). The term "potentially ragged tensor" may be used to refer to a tensor that might be either a Tensor or a RaggedTensor. The ragged-rank of a Tensor is zero.

## Documenting RaggedTensor Shapes

When documenting the shape of a RaggedTensor, ragged dimensions can be indicated by enclosing them in parentheses. For example, the shape of a 3-D RaggedTensor that stores the fixed-size word embedding for each word in a sentence, for each sentence in a batch, could be written as `[num_sentences, (num_words), embedding_size]`. The parentheses around `(num_words)` indicate that dimension is ragged, and that the length of each element list in that dimension may vary for each item.

## Component Tensors

Internally, a RaggedTensor consists of a concatenated list of values that are partitioned into variable-length rows. In particular, each RaggedTensor consists of:

- A values tensor, which concatenates the variable-length rows into a

flattened list. For example, the values tensor for

`[[3, 1, 4, 1], [], [5, 9, 2], [6], []]` is `[3, 1, 4, 1, 5, 9, 2, 6]`.

- A row\_splits vector, which indicates how those flattened values are

divided into rows. In particular, the values for row `rt[i]` are stored

in the slice `rt.values[rt.row_splits[i]:rt.row_splits[i+1]]`.

A values tensor, which concatenates the variable-length rows into a flattened list. For example, the values tensor for

`[[3, 1, 4, 1], [], [5, 9, 2], [6], []]` is `[3, 1, 4, 1, 5, 9, 2, 6]`.

A `row_splits` vector, which indicates how those flattened values are divided into rows. In particular, the values for row `rt[i]` are stored in the slice `rt.values[rt.row_splits[i]:rt.row_splits[i+1]]`.

## Example:

### ## Alternative Row-Partitioning Schemes

In addition to `row_splits`, ragged tensors provide support for five other row-partitioning schemes:

- `row_lengths`: a vector with shape `[nrows]`, which specifies the length

of each row.

- `value_rowids` and `nrows`: `value_rowids` is a vector with shape

`[nvals]`, corresponding one-to-one with values, which specifies each value's row index. In particular, the row `rt[row]` consists of the values `rt.values[j]` where `value_rowids[j]==row`. `nrows` is an integer scalar that specifies the number of rows in the `RaggedTensor`. (`nrows` is used to indicate trailing empty rows.)

- `row_starts`: a vector with shape `[nrows]`, which specifies the start

offset of each row. Equivalent to `row_splits[:-1]`.

- `row_limits`: a vector with shape `[nrows]`, which specifies the stop

offset of each row. Equivalent to `row_splits[1:]`.

- `uniform_row_length`: A scalar tensor, specifying the length of every

row. This row-partitioning scheme may only be used if all rows have the same length.

`row_lengths`: a vector with shape `[nrows]`, which specifies the length of each row.

`value_rowids` and `nrows`: `value_rowids` is a vector with shape `[nvals]`, corresponding one-to-one with values, which specifies each value's row index. In particular, the row `rt[row]` consists of the values `rt.values[j]` where `value_rowids[j]==row`. `nrows` is an integer scalar that specifies the number of rows in the `RaggedTensor`. (`nrows` is used to indicate trailing empty rows.)

`row_starts`: a vector with shape `[nrows]`, which specifies the start offset of each row. Equivalent to `row_splits[:-1]`.

`row_limits`: a vector with shape `[nrows]`, which specifies the stop offset of each row. Equivalent to `row_splits[1:]`.

`uniform_row_length`: A scalar tensor, specifying the length of every row. This row-partitioning scheme may only be used if all rows have the same length.

Example: The following ragged tensors are equivalent, and all represent the nested list `[[3, 1, 4, 1], [], [5, 9, 2], [6], []]`.

## Multiple Ragged Dimensions

RaggedTensors with multiple ragged dimensions can be defined by using a nested RaggedTensor for the values tensor. Each nested RaggedTensor adds a single ragged dimension.

The factory function `RaggedTensor.from_nested_row_splits` may be used to construct a RaggedTensor with multiple ragged dimensions directly, by providing a list of `row_splits` tensors:

## Uniform Inner Dimensions

RaggedTensors with uniform inner dimensions can be defined by using a multidimensional Tensor for values.

## Uniform Outer Dimensions

RaggedTensors with uniform outer dimensions can be defined by using one or more RaggedTensor with a `uniform_row_length` row-partitioning tensor. For example, a RaggedTensor with shape `[2, 2, None]` can be construct